

pgx Driver-Fault Injection Coverage Push — REAL RUN evidence (§11.4.83)

Run date: 2026-06-23 **Run host:** nezha.local (Linux x86_64, go1.26.2, rootless podman 5.7.1 + podman-compose) **Driver host:** mistborn (macOS) — go1.26.2 but NO container runtime; the fault tests were authored + gofmt/vet/compile-checked on mistborn, then rsynced to nezha's ~/helix_ota_itest staging tree (the same tree the integration coverage push used) where go1.26 AND rootless podman both exist, and run there.

Goal

Close the honest pgx driver-fault coverage gap the integration coverage push (docs/qa/20260623-integration-coverage-push/, store 92.4% / rollout 87.3%) flagged: the residual sub-100% lines were uniformly **driver-level fault branches** (mid-query connection kill, rows.Scan/rows.Err() error returns, pool.Query connection-failure, tx Begin/Exec/Commit failure) that are genuinely unreachable against a healthy booted DB. These require **fault injection** to trigger — not a contrived test.

Fault mechanism built — in-process TCP fault-injection proxy

faultproxy_test.go (one per package, kept per-package per §11.4.28 decoupling). A tiny in-process TCP proxy listens on an ephemeral 127.0.0.1:0 port, forwards bytes in both directions to the REAL brick-booted Postgres (port 55432 / 55445), and on Fault() (idempotent) **closes every live connection** (a hard mid-query TCP kill) AND **refuses every new connection** thereafter. The test points a pgxpool DSN at the proxy, runs a healthy warm-up query, then arms the fault and asserts the production pgx methods return a **clean error (never a panic)** — exercising the exact connection-fault branches. This is the standard in-process toxiproxy-equivalent pattern; it is NOT a container runtime, so it is faithful to §11.4.74 (do not reimplement orchestration — the proxy sits in front of the real brick-booted Postgres, which is still booted/torn-down via digital.vasic.containers).

Tests added (4 files, all //go:build integration): - internal/store/faultproxy_test.go + internal/store/postgres_fault_integration_test.go — TestPostgresDriverFaults_Integration (3 subtests). - internal/rollout/

```
faultproxy_test.go + internal/rollout/  
postgres_fault_integration_test.go —  
TestPostgresStoreDriverFaults_Integration (3 subtests).
```

REAL result — ALL PASS, with measured coverage delta

```
ok  internal/store      33.166s  coverage: 93.5% of statements  
STORE_EXIT=0  
ok  internal/rollout   21.299s  coverage: 91.5% of statements  
ROLLOUT_EXIT=0
```

Package	Integration-push baseline	AFTER fault tests	Δ
internal/store	92.4%	93.5%	+1.1pp
internal/rollout	87.3%	91.5%	+4.2pp

Determinism (§11.4.50): both suites ran GREEN across multiple independent –count=1 invocations during this session (store fault-only, rollout fault-only, and the two combined full-suite runs) — identical PASS.

Driver-fault branches now covered (REAL captured faults)

store (store_fault_branches_v.txt) — every method below surfaced a CLEAN driver error (unexpected EOF / connection reset by peer), no panic: - pool.Query / QueryRow connection-failure return branch: ListDevices (86.4→**95.5%**), ListReleases (→90.9%), ListActiveDeployments (→90.9%), LatestRelease (→90.0%), TelemetryEventCounts (→91.7%), DeviceStateCounts (→92.3%), ListGroups (→90.9%), ListProjects (→90.9%), ListAudit, GetDevice (→100%), GetArtifact (→91.7%), and the Exec-error path of CreateDevice/AppendTelemetry. - **mid-iteration rows.Err()**: the proxy was armed *during* a live streaming cursor over 500 seeded rows — mid-iteration rows.Err captured cleanly after 132 rows: unexpected EOF. This is the genuine in-cursor kill landing inside the read loop, exactly the if err := rows.Err(); err != nil branch the production List* loops guard. - **pool-acquire failure**: a lazy pool faulted before first use → Query/QueryRow acquire-fail return.

rollout (rollout_fault_branches_v.txt): - Load QueryRow.Scan / Query connection-failure (Load 79.2→**91.7%**). - Save tx Begin/Exec failure (Save 73.3→**86.7%**) — connection_failure case. - **mid-transaction** kill: Save observed mid-transaction driver fault (attempt 0): unexpected EOF — the fault fired concurrently *inside* a 200-phase Save transaction (Begin + DELETE + 200 INSERTs + Commit). - pool-acquire failure for both Load and Save.

Branches still genuinely unreachable by this proxy (honest §11.4.6)

The fault proxy closes **connection-fault** branches. The following residual sub-100% lines are a DIFFERENT class and are NOT closeable by a connection kill — stated as fact, not bluffed shut:

- **json.Marshal error path** — jsonb0f 75% (store L68–71), nullTime 66.7% (L845–846): json.Marshal of a map[string]string / time.Time **cannot fail** in Go. Structurally unreachable defensive code.
- **json.Unmarshal of DB-stored JSONB error** — scanDevice L161, ListAudit L706, ListRollbacks L808: the DB only ever holds JSONB the code itself wrote, so the unmarshal-error branch is unreachable without injecting corrupt JSONB directly into the table (out of scope — not a driver fault).
- **Migrate Exec-error** — store L52–53 (66.7%), rollout L46–47 (66.7%): a DDL failure on the valid embedded schema. Migrate runs on the healthy seed pool by design (you cannot migrate a faulted connection), so the proxy cannot reach it.
- **NewPostgresStore ping-error** (rollout L31) — needs a bad-DSN unit, not a connection fault; the fault tests use *FromPool by design.
- **rollout Save phase-insert tx.Exec error (L84–85) and Load phases-cursor L116/L123** — these ARE connection-fault branches and are reachable, but landing the kill at that *exact* statement is timing-probabilistic. The mid-transaction subtest hit the tx path on attempt 0 this run; it is honestly written to t.Skip (not fail) if the timing window misses in 8 attempts, with the deterministic connection_failure case covering the Save error path.

The goal was the **reachable driver-fault branches**, not a contorted 100%.

Cleanup (§11.4.14 — PROJECT-SCOPED)

- Boot + teardown via the digital.vasic.containers brick's own mgr.Shutdown (compose down scoped to postgres.compose.yml / postgres-rollout.compose.yml) on t.Cleanup — **never** an ancestor=filter (the prior incident).
- Container baseline captured BEFORE the run: **63** (cleanup_project_scoped.txt). Post-run count: **63** (exact match).
comm -13 baseline after = **empty** — no container from this run remains. No helix_ota / deploy_postgres / port-5543x / port-5544x container present. Other projects' 63 containers were NOT touched.

Evidence files

- integration_fault_run.log — consolidated store+rollout final run (both EXIT=0, both coverages)
- store_fault_branches_v.txt — -v per-method clean-error capture (store)
- rollout_fault_branches_v.txt — -v per-method clean-error capture (rollout)
- coverage_func_fault.txt — go tool cover -func full per-function breakdown (both pkgs)
- cov_store_fault.out / cov_rollout_fault.out — raw coverprofiles

- `cleanup_project_scoped.txt` — baseline==after container accounting